Implementation of Direction-Based Adaptive Beamforming and Spectral Post-Filtering for Noise Suppression on FPGA

Anonymous CVPR submission

Paper ID 10

Abstract

Spatial interference in communication systems results in loss of transmitted information. Beamforming addresses this by using a sensor array to filter out interference from unwanted directions, based on constructive and destructive combinations of signals. This project aims to implement hardware-feasible direction-based adaptive beamforming, followed by spectral post-filtering to suppress residual noise from the beamformed output, targeted for FPGA implementation. Beamforming approaches considered include MVDR, LCMV, and the Generalized Sidelobe Canceller, while noise-reduction approaches considered include Spectral Subtraction, Spectral Gating, Wiener Filtering, and MMSE/Log-MMSE estimation. This report reviews these methods, and they will be compared based on their noise-suppression performance, effect on signal quality, and computational cost, to guide the choice of methods best suited for hardware implementation. The selected design will first be built on the FPGA using recorded audio files, before moving to live microphones and speakers.

1. INTRODUCTION

A single receiver cannot distinguish between signals arriving from different spatial directions, so the received signal is corrupted by directional interference and noise. A uniform linear array of sensors overcomes this by utilizing the phase difference of an incoming wavefront among the different sensor elements. Combining signals from a desired direction constructively while suppressing others is the basic principle underlying beamforming.

Beamforming techniques are broadly grouped into classical and adaptive methods. Classical beamforming applies a fixed set of weights, determined by array geometry, in order to cause constructive interference of signals coming from a known direction. Common methods include Delay-and-Sum, Weighted Delay-and-Sum.

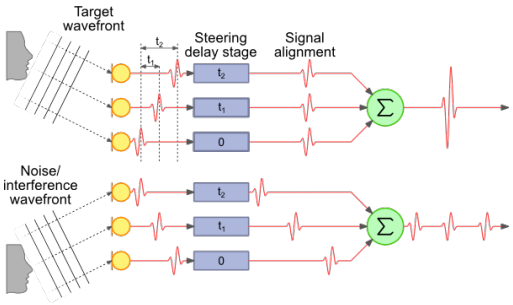


Figure 1. A weighted delay and sum beamformer. Source

In contrast to classical beamforming, which does not adapt weights according to changing signal environments, adaptive beamforming updates array weights based on the received signal statistics, allowing the array to track interference and noise conditions that change over time. This allows adaptive beamforming techniques to achieve effective spatial noise cancellation. MVDR (Capon), LCMV, and the Generalized Sidelobe Canceller (GSC) fall under this category.

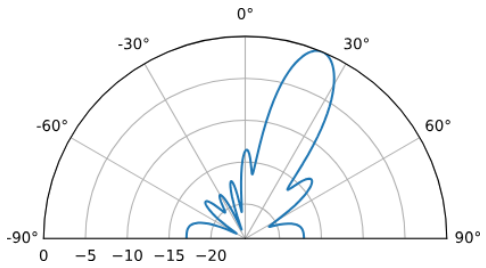


Figure 2. Beam pattern of a linear array in polar form. The angular axis is the direction of arrival and radial axis is the normalized array gain (dB) relative to the peak. The largest main lobe points in the look-direction while side lobes represent signals leaking in. Dips between lobes are spatial nulls. Source

The above mentioned adaptive beamforming methods are effective against directional interference, such as noise or interfering sources arriving from a specific angle relative

to the array, but are not designed to address diffuse, non-directional noise such as background hiss or ambient room noise with no single point of origin. These beamformers are correspondingly reported to give only limited SNR improvement against diffuse noise, with gains scaling with the number of microphones rather than the deep nulling achievable against a coherent point source [15]. Cheng *et al.* [5] explain this by pointing out that at low frequencies, the array cannot focus as sharply, and the noise picked up by different microphones starts to look nearly identical, leaving these beamformers with little spatial difference to work with in order to tell noise apart from the desired signal. In their tests, beamforming alone gave only a 0.33 dB improvement in signal to noise ratio, barely noticeable, while adding a post-filtering stage afterward pushed that improvement up to 6.90 to 9.41 dB, roughly 20 to 28 times better.

Once the beamformer isolates the desired signal from directional interference, a single-channel residual signal remains, still corrupted by non-directional ambient noise. Spectral post-filtering addresses this residual signal by utilizing differences between signal and noise statistics in the frequency domain. Noise suppression methods estimate the noise spectrum, typically during intervals where the desired signal is largely absent, and suppress it from the observed spectrum. Spectral Subtraction, Spectral Gating, Wiener Filtering, and MMSE/Log-MMSE estimation fall under this approach.

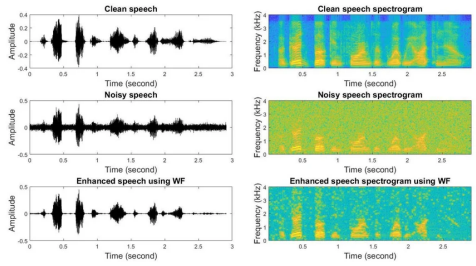


Figure 3. Noise reduction using Wiener Filter. Source

Beyond the algorithmic behavior of these techniques, real-time implementation introduces constraints absent from purely software treatments, such as finite arithmetic precision, limited hardware resources, and strict latency budgets. These constraints shape which methods are practically realisable on hardware. This project addresses direction-based adaptive multichannel beamforming of narrowband signals for the suppression of directional interference, followed by reduction of residual ambient noise using spectral post-filtering, targeted for FPGA implementation.

2. LITERATURE REVIEW

2.1. Adaptive Beamforming Methods

2.1.1 LCMV and MVDR

One of the foundational approaches to adaptive beamforming was presented by Frost [10], who formulated the problem as a linearly constrained optimization that enforces a desired response in a specified look direction and frequency band. It does not require any prior knowledge of the input-signal statistics, unlike several earlier approaches. In the original formulation each sensor is followed by a tapped delay line, enabling the processing of broadband signals. However, for the narrowband case considered in our project, they are omitted and each sensor is represented by a single complex spatial weight. Hence, the resulting Linearly Constrained Minimum Variance (LCMV) problem is defined as

$$\begin{aligned} \underset{\mathbf{w}}{\text{minimize}} \quad & \mathbf{w}^H \mathbf{R}_{xx} \mathbf{w} \\ \text{subject to} \quad & \mathbf{C}^H \mathbf{w} = \mathbf{f}, \end{aligned} \tag{1}$$

where $\mathbf{w}$ is the vector of array weights, $\mathbf{R}_{xx}$ is the spatial covariance matrix of the received signals, $\mathbf{C}$ is the constraint matrix, and $\mathbf{f}$ contains the desired responses corresponding to each constraint.
For example, if θ_s denotes the direction of the desired signal and θ_i denotes the direction of a primary interferer, the beamformer can be constrained to have unit response in the desired direction and zero response in the interference direction. This can be expressed as

$$\mathbf{C}^H \mathbf{w} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \tag{2}$$

where

$$\mathbf{C} = \begin{bmatrix} \mathbf{a}(\theta_s) & \mathbf{a}(\theta_i) \end{bmatrix}. \tag{3}$$

The general optimization problem in Eq. (1) is derived using the principle that if the desired signal is held to a fixed response, minimizing the total output power is equivalent to minimizing the output power of noise alone. An important assumption made is that the non-look direction noise is uncorrelated to the signal.
The covariance matrix is generally unknown a priori and is therefore estimated from received snapshots. The estimated covariance matrix is then used to compute the beamforming weights.

A special case of Eq. (1) is the Minimum Variance Distortionless Response (MVDR) beamformer, in which the constraint set reduces to a single unity-gain constraint in the look direction θ_s:

$$\begin{aligned} \underset{\mathbf{w}}{\text{minimize}} \quad & \mathbf{w}^H \mathbf{R}_{xx} \mathbf{w} \\ \text{subject to} \quad & \mathbf{a}^H(\theta_s) \mathbf{w} = 1, \end{aligned} \tag{4}$$

which yields the closed-form solution

$$\mathbf{w}_{\text{MVDR}} = \frac{\mathbf{R}_{xx}^{-1}\,\mathbf{a}(\theta_s)}{\mathbf{a}^H(\theta_s)\,\mathbf{R}_{xx}^{-1}\,\mathbf{a}(\theta_s)}. \tag{5}$$

A limitation of Frost's LCMV beamformer is its sensitivity to small errors in the physical sensor array, such as sensor position, phase, or gain mismatches. To tackle this, Cox *et al.* [6] presented an improved recursive algorithm which includes multiple linear equality constraints and a quadratic inequality constraint. The uncorrelated errors in the sensors were modeled as white noise.

White noise gain (WNG) measures the gain of the array against uncorrelated spatially white noise and is used as a measure of the robustness to physical imperfections in the array [6]. It is defined as:

$$G_{\text{WNG}} = \frac{\left|\mathbf{w}^H\mathbf{a}(\theta_s)\right|^2}{\mathbf{w}^H\mathbf{w}}. \tag{6}$$

Under the unit-response constraint $\mathbf{a}^H(\theta_s)\mathbf{w} = 1$, this becomes

$$G_{\text{WNG}} = \frac{1}{\mathbf{w}^H\mathbf{w}}. \tag{7}$$

Therefore, $G_{\text{WNG}} \geq \delta^2$ is equivalent to the quadratic inequality constraint

$$\mathbf{w}^H\mathbf{w} \leq \frac{1}{\delta^2}. \tag{8}$$

As seen above, the array's sensitivity to uncorrelated noise decreases as WNG increases.

Two steps that dominate the hardware implementation cost of the above mentioned adaptive beamforming methods are forming the covariance matrix $\mathbf{R}_{xx}$ from the received snapshots, and inverting it to obtain the weights in Eq. (5). Direct inversion is $O(N^3)$ and involves division and square-root operations that are expensive and numerically sensitive in fixed-point arithmetic.

To address the cost of covariance matrix formulation and inversion in hardware Wang *et al.* [19] proposed a method with computational complexity of the order $O(N^2)$.The covariance matrix is first approximated by a Toeplitz matrix, exploiting the fact that, for a uniformly spaced array, the spatial correlation is approximately determined by the separation between sensors. The resulting structured matrix can then be decomposed efficiently using Cholesky decomposition with the Schur algorithm, reducing the computational complexity of the matrix operations. Though this work targets large arrays and an efficient high-throughput implementation, its treatment of low complexity operations is directly relevant to this project.

A parameterized, fixed point FPGA architecture was presented by Llamocca and Aloi [14], for a custom implementation of Frost's linearly constrained adaptive beamformer. The method presented implements complex weighting, accumulation and constraint update operations. Distributed arithmetic is used to reduce resource usage. The paper shows that optimally chosen fixed point representation can yield high accuracy while reducing resource dependence.

This provides a useful hardware-design reference for our implementation, particularly for selecting fixed-point precision and validating the beamformed output against a floating-point reference.

Chen *et al.* [4] note that conventional matrix inversion methods, such as Gauss–Jordan, QR, and LU decomposition, have high computational costs. The diagonal elements of the sample covariance matrix are presented to be substantially larger in absolute value than the off-diagonal elements. Hence, they play a larger role in determining the eigen-vectors and eigen-values, which govern the beamforming weights. Therefore, as a suitable approximation, off-diagonal elements are discarded, reducing the inversion in Eq. (5) to M independent reciprocal operations. Though FPGA implementation was not covered in the paper, on hardware, these can be implemented as parallel reciprocal operations, possibly decreasing resource usage.

2.1.2 Generalized Sidelobe Canceller (GSC)

The Generalized Sidelobe Canceller was proposed by Griffiths and Jim [11] as a way of turning the constrained beamforming problem of MVDR/LCMV into an unconstrained one. The received signal at the array is modelled as

$$\mathbf{x}(n) = \mathbf{a}(\theta_s)\,s(n) + \mathbf{n}(n), \tag{9}$$

where $\mathbf{x}(n)$ is the vector of sensor signals, $\mathbf{a}(\theta_s)$ is the steering vector for the desired signal arriving from direction θ_s, $s(n)$ is the desired signal, and $\mathbf{n}(n)$ is interference and noise. The GSC splits the array output into two branches. A fixed, quiescent beamformer keeps the response toward the desired direction unchanged, giving an output

$$y_q(n) = \mathbf{w_q}^H\mathbf{x}(n), \tag{10}$$

where $\mathbf{w_q}$ is the beamforming weight vector. Alongside this, a blocking matrix $\mathbf{B}$ satisfying

$$\mathbf{B}^H\mathbf{a}(\theta_s) = \mathbf{0} \tag{11}$$

removes the desired signal and passes only interference and noise into an adaptive branch,

$$\mathbf{u}(n) = \mathbf{B}^H\mathbf{x}(n). \tag{12}$$

An adaptive filter estimates this unwanted component,

$$y_a(n) = \mathbf{g}^H(n)\,\mathbf{u}(n), \tag{13}$$

which is subtracted from the fixed branch's output to give the final signal,

$$y(n) = y_q(n) - y_a(n). \tag{14}$$

Because this avoids the direct matrix inversion needed in an explicit LCMV/MVDR solution, the problem reduces to a standard adaptive-filtering task, using LMS, NLMS, or RLS, which mainly involves additions, subtractions, and multiplications and is therefore easier to map onto FPGA hardware than large matrix operations.

The main limitation of the basic GSC comes from the blocking matrix. If the assumed direction of arrival does not exactly match the true source direction, due to sensor-position errors, gain mismatch, or a moving source, the blocking matrix can no longer fully null the desired signal. Part of it then leaks into the adaptive branch, where the filter treats it as interference and cancels it, so the GSC ends up removing some of the signal it was meant to preserve. Hoshuyama *et al.* [13] studied this problem and proposed the Robust GSC (RGSC), combining two changes: coefficient-constrained adaptive filters in the blocking matrix, which let it adapt within a limited range to correct direction errors of up to about 20° without drifting onto an interferer, and norm-constrained adaptive filters in the canceller stage, which cap the filter's coefficient strength so leftover leakage cannot be cancelled too aggressively. The norm constraint is

$$\|\mathbf{g}(n)\| \le \gamma, \tag{15}$$

where γ is the maximum permitted coefficient norm. Their listening tests improved from 2.8/5 for the conventional GSC to 3.8/5 with RGSC, and the method achieved over 30 dB of interference suppression in anechoic conditions and about 19 dB of noise suppression in a real reverberant room, at roughly twice the multiplication count of a fixed beamformer [13]. More recently, Hong *et al.* [12] proposed a robust RLS-based GSC that runs a network of GSCs pointed at several candidate directions instead of one fixed assumption, combining them through spatial regularization, with the RLS part implemented using a QR-decomposition structure for hardware efficiency, showing that DOA-mismatch robustness for GSC is still being actively developed.

On the hardware side, Zhao *et al.* [21] implemented the GSC on an FPGA for an 8-sensor array. They replaced the usual fixed blocking matrix with a DCT-based one, which decorrelates the sensor signals and speeds up convergence. The DCT used is

$$X_k = \sum_{n=0}^{N-1} x_n \cos\left[\frac{\pi}{N}\left(n+\frac{1}{2}\right)k\right], \tag{16}$$

where N is the transform length, x_n is the input sequence, and X_k is the k-th DCT coefficient. They also removed the division step from the LMS weight update by quantizing the normalization factor into a small number of levels, so it could be computed using canonical signed digit (CSD) shifts and additions instead of a hardware divider. Their fixed-point Verilog design on a Xilinx Virtex-7 (XC7VX330T) used at least 21% less hardware and ran at least 26.85% faster than a conventional divider-based version, with similar convergence behavior [21].

Together, these works show that the GSC avoids the large matrix inversions required by MVDR/LCMV, but its accuracy depends on how well the blocking matrix matches the true source direction, a problem addressed through added adaptive constraints [13] or spatial regularization [12] at some extra computational cost.

2.2. Spectral Post-filtering Methods

2.2.1 Spectral Subtraction

Spectral subtraction, proposed by Boll [3] in 1979, is based on a simple observation that the noise and the speech in a recording add together, so if we know roughly how much noise is present at each frequency, we can remove that amount from the signal. If $s(n)$ is the clean speech and $d(n)$ is the background noise, the microphone signal is

$$x(n) = s(n) + d(n). \tag{17}$$

The signal is first divided into short frames of $N = 256$ samples (about 32 ms at 8 kHz), each overlapping the previous one by half. Every frame is multiplied by a Hanning window,

$$w(n) = 0.5 - 0.5\cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \le n \le N-1, \tag{18}$$

which fades the frame smoothly to zero at both ends. This avoids sharp edges that would spread energy into the wrong frequencies, and it lets the overlapping frames add back to the original signal at the end. An FFT of each windowed frame gives $X(k)$ at frequency bin k, which we split into a magnitude $|X(k)|$ and a phase $\theta(k)$.

The noise estimate can be obtained from an initial noise-only segment or from detected noise-dominant frames.. The magnitudes of M noise-only frames are averaged to give the expected noise magnitude at each frequency,

$$\mu(k) = \frac{1}{M}\sum_{i=0}^{M-1} |X_i(k)|, \tag{19}$$

where Boll used about one third of a second of noise-only signal at the start [3]. This estimate is then subtracted from the magnitude of every new frame,

$$|\hat{S}(k)| = |X(k)| - \mu(k). \tag{20}$$

Because $\mu(k)$ is only an average, the noise in a particular frame can be smaller than $\mu(k)$, which makes the result negative. A magnitude cannot be negative, so such values are set to zero (half-wave rectification),

$$|\hat{S}(k)| = \max\big(|X(k)| - \mu(k),\, 0\big). \tag{21}$$

The noisy phase is reused without change, since the ear is far less sensitive to phase errors than to magnitude errors. The cleaned spectrum is

$$\hat{S}(k) = |\hat{S}(k)|\, e^{j\theta(k)}, \tag{22}$$

and an inverse FFT followed by overlap-add of the frames gives the enhanced waveform.

The method is attractive because it works on the waveform itself as a preprocessing step. Earlier options were noise-cancelling microphones, which give little reduction above 1 kHz, or modifying each voice processor internally, which is costly, so Boll's approach lets systems placed after it stay unchanged [3]. It needs a single microphone and about as much computation as a fast convolution.

Its main weakness is the leftover noise after rectification. In each frame, some frequencies keep small random peaks and others are cut to zero, so the residual sounds like faint tones switching on and off about every 20 ms. This is known as musical noise [3]. Boll reduced it by replacing each value with the minimum over three neighbouring frames, at the cost of one frame of delay and extra memory. Berouti *et al.* [2] instead subtract slightly more than the estimated noise and never allow the result to fall below a small fraction of the noisy signal,

$$|\hat{S}(k)|^2 = \max\big(|X(k)|^2 - \alpha\,\mu(k)^2,\; \beta\,|X(k)|^2\big), \tag{23}$$

where $\alpha \geq 1$ is the over-subtraction factor and $0 < \beta \ll 1$ is the spectral floor. Both must be tuned, and an FPGA study found $\beta = 0.01$ worked best for its system [7]. Two further limits come from the assumptions behind (19). The noise must stay roughly steady, and if it changes faster than the estimate can be updated, the reduction becomes less effective and can remove speech where little noise is present. A speech activity detector is also needed to decide when the estimate may be updated [3].

Spectral subtraction has been implemented on hardware several times. Bahoura and Ezzaidi [1] built it on an FPGA using a high-level programming tool, showing that the FFT, magnitude, subtraction and inverse FFT blocks fit a standard FPGA design flow. In an FPGA-based speech recognition system [7], the hardware version reached 80.28% average correlation with clean speech, against 83.7% for the MATLAB model, so fixed-point arithmetic cost a small amount of accuracy. A multi-band, phase-aware version has also been realised on a Virtex-6 FPGA [17]. Boll's original design already suits hardware, because real-valued speech lets two frames share one FFT, halving storage, and no zero padding is required [3]. The algorithm needs no matrix inversion, and its only demanding blocks are the FFT and IFFT, for which vendor cores exist, and the magnitude and phase computation, which a CORDIC block can perform using only shifts and additions. The window values in (18) can be stored in a ROM. The speech/silence decision can be made by comparing against a precomputed threshold, avoiding a logarithm and a division. The main points to watch are the word length of the fixed-point FFT and the added memory and delay if the three-frame minimum is used.

In our system, the beamformer output is a single channel, which is the setting spectral subtraction was designed for, so it needs no knowledge of the array geometry or the direction of arrival. It adds little cost on top of the resource-heavy beamforming stage, and its FFT may be shared with a frequency-domain beamformer. It targets the diffuse background noise that beamforming leaves behind [5, 15], and it gives a low-cost baseline for measuring how much a post-filtering stage improves the output.

2.2.2 Spectral Gating

Spectral subtraction removes the same average amount of noise at every frequency and, as noted above, produces musical noise and assumes steady noise. Spectral gating instead decides, for each time-frequency point, whether to keep or attenuate it, like a noise gate applied per band. This idea was developed into a practical algorithm by Sainburg and Zorea [16] in the *Noisereduce* package which is a free Python library. It needs no training data, only an estimate of the noise.

The signal is first converted to a spectrogram $X(k, l)$ with a short-time Fourier transform, where k is the frequency bin and l is the frame. The level of each point is

$$L(k, l) = 20 \log_{10} |X(k, l)|. \tag{24}$$

The noise is measured from a noise-only clip, or from quiet parts of the recording. For each frequency, we find the mean level $\mu_d(k)$ and the standard deviation $\sigma_d(k)$ of the noise. The gate threshold is set a little above the average noise level,

$$T(k) = \mu_d(k) + \gamma\, \sigma_d(k), \tag{25}$$

where γ controls how strict the gate is.

Each point is then compared with the threshold. Points above it are kept, and points below it are reduced. This gives the output

$$\hat{S}(k, l) = G(k, l)\, X(k, l), \tag{26}$$

where $G(k, l)$ is 1 for points above the threshold and a smaller value for points below it. The noisy phase is kept, and an inverse transform gives the cleaned signal.

The threshold can be fixed for the whole recording (stationary mode) or updated over time (non-stationary mode) [16]. The second mode can follow noise that changes, which spectral subtraction cannot do. The main weaknesses are that a high threshold removes weak consonants, and that a sudden change in the gate can cause musical noise.

2.2.3 Wiener Filtering, MMSE-STSA and log-MMSE

Noisy speech is modeled as

$$y(t) = x(t) + d(t), \tag{27}$$

where $x(t)$ is the clean speech we want and $d(t)$ is additive noise. Speech is quasi-stationary over 20–40 ms, so the signal is cut into short overlapping frames. Each frame is windowed and Fourier-transformed, giving one complex number per frequency bin k:

$$\begin{aligned} Y_k &= R_k\, e^{j\theta_k} \quad \text{(noisy)}, \\ X_k &= A_k\, e^{j\alpha_k} \quad \text{(clean speech)}, \end{aligned} \tag{28}$$

and D_k for the noise. R_k and A_k are amplitudes (how much energy is at frequency k), and θ_k and α_k are phases (timing information). The goal of every method here is to recover the clean component from the noisy Y_k, which is the only thing we can observe. Each method does this by multiplying Y_k by a gain between 0 and 1: a gain near 1 keeps a bin, and a gain near 0 suppresses it. The methods differ only in how they compute that gain.

Speech and noise are modeled as independent, zero-mean Gaussian random variables at each frequency [8]. The justification is the central limit theorem: each Fourier coefficient is a weighted sum of many samples, and sums of many small random contributions tend toward a Gaussian. For the amplitude a_k, this gives

$$p(a_k, \alpha_k) = \frac{a_k}{\pi\, \lambda(k)} \exp\!\left(-\frac{a_k^2}{\lambda(k)}\right), \tag{29}$$

where $\lambda(k) = \lambda_x(k)$ for speech and $\lambda(k) = \lambda_d(k)$ for noise, with

$$\lambda_x(k) = E\{|X_k|^2\}, \quad \lambda_d(k) = E\{|D_k|^2\} \tag{30}$$

being the average speech power and average noise power at bin k. Under this model, one number per bin fully describes the statistics of speech, and one describes the noise. Small amplitudes are common and large spikes are rare, with λ setting the scale. The phase does not appear in the formula because it is assumed uniformly random. Two SNR terms are used throughout: the a priori SNR, which is the expected long-term SNR of bin k, and the a posteriori SNR, which is how strong bin k looks in the current frame,

$$\xi_k = \frac{\lambda_x(k)}{\lambda_d(k)}, \qquad \gamma_k = \frac{R_k^2}{\lambda_d(k)}. \tag{31}$$

The Wiener filter [20] looks for the linear estimate $\hat{X}_k = G\, Y_k$ that minimizes the mean-square error on the whole complex value,

$$\min_G \; E\{\, |X_k - \hat{X}_k|^2 \,\}. \tag{32}$$

$$\hat{X}_k = G_w(\xi_k)\, Y_k, \qquad G_w(\xi_k) = \frac{\xi_k}{1+\xi_k}. \tag{33}$$

The gain behaves sensibly at both extremes: for $\xi_k = 9$ (clean bin), $G_w = 0.9$ and the bin is almost fully kept; for $\xi_k = 1$ (speech and noise equal), $G_w = 0.5$; for $\xi_k = 0.1$ (noise-dominated), $G_w \approx 0.09$ and the bin is heavily suppressed. The gain is real-valued, so it does not change the phase. The enhanced component keeps the noisy phase θ_k, and the amplitude estimate is

$$\hat{A}_k = G_w(\xi_k)\, R_k. \tag{34}$$

Two properties of this filter matter for what follows. First, the gain depends only on ξ_k, the long-term expected SNR, and ignores γ_k, so it does not react to how the current frame actually looks. Second, it was optimized for the complex value, so the amplitude estimate is only the modulus of that result, not something optimized for amplitude.

Hearing depends far more on the short-time spectral amplitude (STSA) than on the phase. Since the Wiener amplitude is only a byproduct, the MMSE-STSA estimator of Ephraim and Malah [8] minimizes the amplitude error directly,

$$\min \; E\{(A_k - \hat{A}_k)^2\} \;\Rightarrow\; \hat{A}_k = E\{A_k \mid Y_k\}. \tag{35}$$

The best estimate is the average clean amplitude given the observation. Evaluating this integral with the Gaussian model above gives $\hat{A}_k = G_{\mathrm{MMSE}}(\xi_k, \gamma_k)\, R_k$, where

$$\begin{aligned} G_{\mathrm{MMSE}} = \Gamma(1.5)\, \frac{\sqrt{v_k}}{\gamma_k}\, e^{-v_k/2} \\ \times \left[(1+v_k)\, I_0\!\left(\tfrac{v_k}{2}\right) + v_k\, I_1\!\left(\tfrac{v_k}{2}\right)\right], \end{aligned} \tag{36}$$

$$v_k = \frac{\xi_k}{1+\xi_k}\, \gamma_k. \tag{37}$$

Γ is the gamma function, and I_0 and I_1 are modified Bessel functions. Unlike Wiener, this gain depends on both ξ_k (prior expectation) and γ_k (current evidence). At high SNR ($\xi_k \gg 1$) it reduces to $\xi_k/(1+\xi_k)$, the Wiener gain, so the two agree when noise is low and differ at low SNR. The paper also shows that reusing the noisy phase is the optimal phase choice, so only the amplitude needs to be enhanced. Speech is not present in every bin of every frame, so [8]

adds q_k, the probability that speech is absent, and multiplies the gain by a confidence factor,

$$\hat{A}_k = \frac{\Lambda}{1+\Lambda}\, G_{\mathrm{MMSE}}(\xi_k, \gamma_k)\, R_k, \qquad \Lambda = \mu_k \frac{e^{v_k}}{1+\xi_k}, \quad \mu_k = \frac{1-q_k}{q_k}. \tag{38}$$

The factor is close to 1 when speech is likely present and close to 0 when it is likely absent.

The log-MMSE estimator (Ephraim and Malah, 1985) [9] keeps the same model but changes the error measure to the log of the amplitude, because loudness perception is roughly logarithmic,

$$\min\ E\{(\log A_k - \log \hat{A}_k)^2\} \quad \Rightarrow\ \hat{A}_k = \exp\big(E\{\ln A_k \mid Y_k\}\big). \tag{39}$$

Written out,

$$\hat{A}_k = \frac{\xi_k}{1+\xi_k}\, \exp\left\{\frac{1}{2}\int_{v_k}^{\infty} \frac{e^{-t}}{t}\, dt\right\} R_k = G_{\mathrm{logMMSE}}(\xi_k, \gamma_k)\, R_k. \tag{40}$$

The integral is the exponential integral, which takes the place of the Bessel functions. It is evaluated by differentiating a moment-generating function at $\mu = 0$. The gain again depends on ξ_k and γ_k, and Jensen's inequality shows it is never larger than the MMSE-STSA gain [9]. The authors did not combine it with the speech-absence extension, because it introduced a low-pass filtering effect on the speech [9].

The true ξ_k is unknown, so it is estimated with the decision-directed approach of [8], with $\alpha \approx 0.98$,

$$\hat{\xi}_k(n) = \alpha\, \frac{\hat{A}_k^2(n-1)}{\lambda_d(k, n-1)} + (1-\alpha)\, P\big[\gamma_k(n) - 1\big], \tag{41}$$

where $P[x] = \max(x, 0)$. The first term reuses the previous frame's cleaned amplitude, and the second uses the current frame's instantaneous SNR. A large α favors the smoothed history over noisy frame-to-frame fluctuations. In [8] the noise power $\lambda_d(k)$ was estimated once from an initial 320 ms noise-only segment, which assumes stationary noise. Both families of estimators therefore depend on accurate estimates of ξ_k and the noise power.

The Wiener gain is a single ratio and is the cheapest of the three to compute [20]. MMSE-STSA uses both ξ_k and γ_k, while Wiener uses only ξ_k, so the two agree at high SNR but MMSE-STSA does better at low SNR, which is exactly where noise suppression matters most [8]. It also has a guaranteed error bound, unlike Wiener, since it was directly built to minimize amplitude error [8]. In practice, listening tests showed MMSE with decision-directed SNR leaves smooth, steady residual noise, unlike the "musical noise" from spectral subtraction [8]. Log-MMSE improves on this further: its gain is proven to always be lower than MMSE-STSA's, which explains its even quieter residual noise without harming the speech itself [9].

For the system in [8], the processing pipeline is: first the signal is framed (256 samples, 192-sample overlap, 8 kHz), then a Hanning window is applied, then the FFT is taken, then ξ_k and γ_k are computed, then the gain is calculated, then the amplitude is multiplied by the gain, then the noisy phase is reattached, then the IFFT is taken, and finally the frames are combined by overlap-add. For MMSE-STSA, Ephraim and Malah used lookup tables of 961 gain samples in their system instead of evaluating the gain function directly. The tables covered the range -15 to $+15$ dB and were reported to add negligible residual noise for input SNRs of -5 to 5 dB [8]. The log-MMSE estimator was run in that same system [9]. The decision-directed estimate of ξ_k uses the previous frame's cleaned amplitude $\hat{A}_k$ [8].

In our system, the Wiener filter is the easiest option to start with. It needs just one division per frequency bin, so it is simple to build on an FPGA and fits well after the beamformer to clean up the noise. MMSE-STSA and log-MMSE work better when the noise is high, and they leave a smoother, less irritating noise behind. But they are harder to build, because they need a lookup table and they also need to remember the previous frame's output. Paper [18] found that MMSE can distort the speech more than Wiener, so the right choice depends on what we care about more: getting the amplitude as accurate as possible, or keeping the speech sounding natural.

3. NEXT STEPS

The next stage of the project is to compare the methods reviewed, choose one design, and build it on the FPGA. We plan to first work with recorded audio files, and then move to live microphones and speakers.

1. **Evaluation.** We will compare the different beamforming methods by how well they block the interfering source. We will compare the post-filtering methods by how much they improve the SNR and how much they distort the signal. We will also check how much better the output is with a post-filter than with beamforming alone.

2. **Hardware feasibility.** For each method, we will estimate how much FPGA logic and memory it needs, how much delay it adds, and whether it needs costly operations such as matrix inversion, division or lookup tables.

3. **Selection.** Using these results, we will choose one

beamforming method and one post-filtering method that give a good balance between quality and hardware cost.

4. **FPGA implementation with audio files.** A noisy multichannel audio file will be given to the board as input, and the processed output will be read back and played or saved as an audio file. This lets us test the hardware without any analog front end.

5. **Live demonstration (future work).** If time permits, we will replace the audio file input with a real microphone array connected to the board, and send the output to a speaker.

References

[1] M. Bahoura and H. Ezzaidi. Implementation of spectral subtraction method on fpga using high-level programming tool. In *Proc. 24th Int. Conf. Microelectronics (ICM)*. IEEE, 2012. 5

[2] M. Berouti, R. Schwartz, and J. Makhoul. Enhancement of speech corrupted by acoustic noise. In *Proc. IEEE ICASSP*, volume 4, pages 208–211, 1979. 5

[3] Steven F. Boll. Suppression of acoustic noise in speech using spectral subtraction. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 27(2):113–120, 1979. 4, 5

[4] Weiheng Chen, Yihuan Zhao, and Jing Gao. Improved Capon beamforming algorithm by using inverse covariance matrix calculation. In *IET International Radar Conference 2013*, pages 1–6, 2013. 3

[5] Ning Cheng, Wen-Ju Liu, Peng Li, and Bo Xu. Microphone array speech enhancement based on a generalized post-filter and a novel perceptual filter. In *Proc. ICSP2008*, pages 370–373. IEEE, 2008. 2, 5

[6] Henry Cox, Robert M. Zeskind, and Mark H. Owen. Robust adaptive beamforming. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 35(10):1365–1376, 1987. 3

[7] De La Salle University. Spectral subtraction speech enhancement integrated to automatic speech recognition system implemented in fpga. Master's thesis, De La Salle University, 2012. 5

[8] Y. Ephraim and D. Malah. Speech enhancement using a minimum mean-square error short-time spectral amplitude estimator. *IEEE Trans. Acoust., Speech, Signal Process.*, ASSP-32(6):1109–1121, Dec. 1984. 6, 7

[9] Y. Ephraim and D. Malah. Speech enhancement using a minimum mean-square error log-spectral amplitude estimator. *IEEE Trans. Acoust., Speech, Signal Process.*, ASSP-33(2):443–445, Apr. 1985. 7

[10] III Frost, Otis Lamont. An algorithm for linearly constrained adaptive array processing. *Proceedings of the IEEE*, 60(8):926–935, 1972. 2

[11] L. J. Griffiths and C. W. Jim. An alternative approach to linearly constrained adaptive beamforming. *IEEE Transactions on Antennas and Propagation*, 30(1):27–34, 1982. 3

[12] J. Hong, Y. Chu, S. C. Chan, Y. Zhao, Y. Zhou, and W. U. Ming. A robust RLS-based generalized sidelobe canceller using spatial regularization. *Digital Signal Processing*, 164:105273, 2025. 4

[13] Osamu Hoshuyama, Akihiko Sugiyama, and Akihiro Hirano. A robust adaptive beamformer for microphone arrays with a blocking matrix using constrained adaptive filters. *IEEE Transactions on Signal Processing*, 47(10):2677–2684, 1999. 4

[14] Daniel Llamocca and Daniel N. Aloi. A reconfigurable fixed-point architecture for adaptive beamforming. In *2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 132–138. IEEE, 2016. 3

[15] Iain A. McCowan and Hervé Bourlard. Microphone array post-filter based on noise field coherence. *IEEE Transactions on Speech and Audio Processing*, 11(6):709–716, 2003. 2, 5

[16] Tim Sainburg and Aviad Zorea. Noisereduce: Domain general noise reduction for time series signals. *arXiv preprint arXiv:2412.17851*, 2024. 5, 6

[17] Suman Samui, Pratik Sahu, Indrajit Chakrabarti, and Soumya K. Ghosh. FPGA implementation of a phase-aware single-channel speech enhancement system. *Circuits, Systems, and Signal Processing*, 36(11):4688–4715, 2017. 5

[18] P. Scalart, J. Vieira Filho, and J. G. Chiquito. On speech enhancement algorithms based on MMSE estimation. In *Proc. Eur. Signal Process. Conf. (EUSIPCO)*, 1996. 7

[19] Shin-Shiang Wang, Yi-Chi Tien, Yin-Tsung Hwang, Jin-Fa Lin, and Guo-Zua Wu. MVDR based adaptive beamformer design and its fpga implementation for ultrasonic imaging. In *2016 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, pages 143–145. IEEE, 2016. 3

[20] N. Wiener. *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*. MIT Press, Cambridge, MA, USA, 1949. 6, 7

[21] W. Zhao, J. Q. Lin, S. C. Chan, and H. K.-H. So. A division-free and variable-regularized LMS-based generalized sidelobe canceller for adaptive beamforming and its efficient hardware realization. *IEEE Access*, 6:64470–64485, 2018. 4